Relational Databases and Python

Today we explore relational databases using Python. We have been using the relational database, Microsoft Access until now. Today we will use sqlite3, which is another relational database.

References:

- https://datatofish.com/create-database-python-using-sqlite3/
- https://www.python-course.eu/sql_python.php

Part 1

We will start by creating a database, `company.db` from scratch. The next block of code will pretty much be the same no matter what.

In [1]:
```
# import the sqlite3 library
import sqlite3

# create a connection object
connection = sqlite3.connect("company.db")

# create a cursor object
cursor = connection.cursor()
```

We now have an empty SQLite database file called company.db, which is saved in the same location as this Jupyter notebook file. Naturally, our next step is to create some tables.

Recall from our earlier exercises, our SQL commands to create tables:

In [2]:
```
# if necessary:
try:
    cursor.execute('DROP TABLE employee')
except:
    pass

sql_command = """
CREATE TABLE employee (
staff_number INTEGER PRIMARY KEY,
fname VARCHAR(20),
lname VARCHAR(30),
gender CHAR(1),
joining DATE,
birth_date DATE);"""

# note:  the above can also be written as this:
# sql_command = "CREATE TABLE employee ( staff_number INTEGER PRIMARY KEY, fname VARCHAR(20), lna
# more info at: https://stackoverflow.com/questions/19479853/why-do-we-need-to-use-3-quotes-while
```

We've saved the SQL command to create an `employee` table to the variable `sql_command`. In order to execute this SQL statement, the following is needed. Think of this as the same as clicking "Run" in Access.

In [3]:
```
cursor.execute(sql_command)
```

Out[3]:
```
<sqlite3.Cursor at 0x202e0f6e940>
```

Note: once a table has been created, you cannot rerun the `CREATE TABLE` SQL command. You will have to drop the table and then you can recreate it. To drop the `employee` table: `cursor.execute("""DROP TABLE employee;""")`

Now that we have created a table, let's populate it. Recall from earlier, we can INSERT INTO the `employee` table using the following SQL statement:

In [4]:
```
# insert data into employee table
sql_command = """INSERT INTO employee (staff_number, fname, lname, gender, birth_date)
    VALUES (123, "Frank", "Schiller", "m", "1955-08-17");"""

# execute SQL command
cursor.execute(sql_command)
```

Out[4]:
```
<sqlite3.Cursor at 0x202e0f6e940>
```

When inserting into a table, you must also commit changes. Think of this as clicking "save" in Access

In [5]:
```
# commit changes to the SQL database
connection.commit()
```

Did anything happen? You can look at the table by running a `SELECT *` query and then "fetching" the results:

In [6]:
```
sql_command = """SELECT * from employee;"""
cursor.execute(sql_command)
```

Out[6]:
```
<sqlite3.Cursor at 0x202e0f6e940>
```

In [7]:
```
results = cursor.fetchall()
print(results)
```

```
[(123, 'Frank', 'Schiller', 'm', None, '1955-08-17')]
```

I know what you're thinking -- this isn't realistic to enter in record by record. Here's a way to do several records at once:

In [8]:
```
staff_data = [ ("345", "William", "Shakespeare", "m", "1961-10-25"),
               ("678", "Wendy", "Wu", "f", "1955-08-17"),
               ("910", "Jane", "Wall", "f", "1989-03-14"),
               ("112", "Stephen", "Rodriguez", "m", "1971-2-25"),
               ("213", "Taylor", "Dahl", "f", "1953-3-02")]
```

We will make use of some Python f strings for text formatting. Quick example below to give you the idea.

In [9]:
```
price = 49
print(f'For only {price} dollars!')
```

```
For only 49 dollars!
```

Now back to the SQL work.

In [10]:
```
for record in staff_data:
    print('Current record is: ',record)
    num, first, last, gender, birthdate = record
    sql_command = f"""INSERT INTO employee (staff_number, fname, lname, gender, birth_date)
    VALUES ("{num}", "{first}", "{last}", "{gender}", "{birthdate}");"""
    print('SQL command to add it to the table is:\n',sql_command,'\n')
    cursor.execute(sql_command)

connection.commit()
```

```
Current record is:  ('345', 'William', 'Shakespeare', 'm', '1961-10-25')
SQL command to add it to the table is:
 INSERT INTO employee (staff_number, fname, lname, gender, birth_date)
    VALUES ("345", "William", "Shakespeare", "m", "1961-10-25");

Current record is:  ('678', 'Wendy', 'Wu', 'f', '1955-08-17')
SQL command to add it to the table is:
 INSERT INTO employee (staff_number, fname, lname, gender, birth_date)
    VALUES ("678", "Wendy", "Wu", "f", "1955-08-17");

Current record is:  ('910', 'Jane', 'Wall', 'f', '1989-03-14')
SQL command to add it to the table is:
 INSERT INTO employee (staff_number, fname, lname, gender, birth_date)
    VALUES ("910", "Jane", "Wall", "f", "1989-03-14");

Current record is:  ('112', 'Stephen', 'Rodriguez', 'm', '1971-2-25')
SQL command to add it to the table is:
 INSERT INTO employee (staff_number, fname, lname, gender, birth_date)
    VALUES ("112", "Stephen", "Rodriguez", "m", "1971-2-25");

Current record is:  ('213', 'Taylor', 'Dahl', 'f', '1953-3-02')
SQL command to add it to the table is:
 INSERT INTO employee (staff_number, fname, lname, gender, birth_date)
    VALUES ("213", "Taylor", "Dahl", "f", "1953-3-02");
```

What did the following lines of code accomplish?

```
num, first, last, gender, birthdate = record
sql_command = f"""INSERT INTO employee (staff_number, fname, lname, gender,
birth_date)
VALUES ("{num}", "{first}", "{last}", "{gender}", "{birthdate}");"""
```

Your answer:

.

Let's run the SELECT * from EMPLOYEE query again and fetch the results. The loop below will display them.

In [11]:
```
sql_command = """SELECT * from employee;"""
cursor.execute(sql_command)
results = cursor.fetchall()
for res in results:
    print(res)
```

```
(112, 'Stephen', 'Rodriguez', 'm', None, '1971-2-25')
(123, 'Frank', 'Schiller', 'm', None, '1955-08-17')
(213, 'Taylor', 'Dahl', 'f', None, '1953-3-02')
(345, 'William', 'Shakespeare', 'm', None, '1961-10-25')
(678, 'Wendy', 'Wu', 'f', None, '1955-08-17')
(910, 'Jane', 'Wall', 'f', None, '1989-03-14')
```

Here is code for another query. Fill in the code needed below it to display the results.

In [12]:
```
cursor.execute('SELECT lname, gender from employee;')
print(cursor.fetchall())
```

```
[('Rodriguez', 'm'), ('Schiller', 'm'), ('Dahl', 'f'), ('Shakespeare', 'm'), ('Wu', 'f'), ('Wal
l', 'f')]
```

I know what you're thinking..."wouldn't it be nice to view this in a pandas dataframe?"

In [13]:
```
import pandas as pd

sql_command = """SELECT * from employee;"""
df = pd.read_sql_query(sql_command, connection)
df
```

Out[13]:

	staff_number	fname	lname	gender	joining	birth_date
0	112	Stephen	Rodriguez	m	None	1971-2-25
1	123	Frank	Schiller	m	None	1955-08-17
2	213	Taylor	Dahl	f	None	1953-3-02
3	345	William	Shakespeare	m	None	1961-10-25
4	678	Wendy	Wu	f	None	1955-08-17
5	910	Jane	Wall	f	None	1989-03-14

Display a pandas data frame for a query that selects last name, gender, and joining from employee.

In [14]:
```
pd.read_sql_query('SELECT lname, gender, joining from employee', connection)
```

Out[14]:

	lname	gender	joining
0	Rodriguez	m	None
1	Schiller	m	None
2	Dahl	f	None
3	Shakespeare	m	None
4	Wu	f	None
5	Wall	f	None

PART I PRACTICE QUESTIONS:

Write the following SQL statements, return the results in a pandas dataframe.

1. Return first and last names of everyone in the `employee` table
2. Return all fields of all males in the `employee` table
3. Return all fields of all females who were born before 1970 in the `employee` table, order in decending by birth date

In [15]:
```
# 1. Return first and last names of everyone in the employee table
'''YOUR CODE HERE'''
sql1 = 'SELECT fname, lname from employee'
df = pd.read_sql_query(sql1,connection)
df
```

Out[15]:

	fname	lname
0	Stephen	Rodriguez
1	Frank	Schiller
2	Taylor	Dahl
3	William	Shakespeare
4	Wendy	Wu
5	Jane	Wall

In [16]:
```
# 2. Return all fields of all males in the employee table
'''YOUR CODE HERE'''
pd.read_sql_query('SELECT * from employee WHERE "gender"="m"',connection)
```

Out[16]:

	staff_number	fname	lname	gender	joining	birth_date
0	112	Stephen	Rodriguez	m	None	1971-2-25
1	123	Frank	Schiller	m	None	1955-08-17
2	345	William	Shakespeare	m	None	1961-10-25

In [17]:
```
# 3. Return all fields of all females who were born before 1970 in the employee table, order in d
'''YOUR CODE HERE'''
sql3 = """

SELECT
    * from employee

WHERE
    "birth_date"<"1970-12-31" and "gender"="f"

ORDER by
    "birth_date" DESC

"""
pd.read_sql_query(sql3,connection)
```

Out[17]:

	staff_number	fname	lname	gender	joining	birth_date
0	678	Wendy	Wu	f	None	1955-08-17
1	213	Taylor	Dahl	f	None	1953-3-02

Finally, to close the database, close out your `connection` and `cursor` objects.

In [18]:
```
connection.close()
```

Part 2

Let's explore another way to enter data into a database -- importing a .csv file

In [19]:
```
# Make the feline.db database (make the connection object)
# Make the cursor object
connection = sqlite3.connect('feline.db')
cursor = connection.cursor()
```

In [20]:
```
# Read in the csv
customer_df = pd.read_csv ('Customers.csv')
```

In [21]:
```
# Insert the values from the csv file into the table 'customer'
customer_df.to_sql('Customers', connection, if_exists='replace', index = False)
```

Out[21]:
```
11
```

Peform the query to see the whole table and put the results into a pandas data frame as before.

In [22]:
```
# View the data
sql_command = """SELECT * from Customers;"""
df = pd.read_sql_query(sql_command, connection)
df.head()
```

Out[22]:

	Customer ID	First Name	Last Name	Email Address	Street Address	City	State	Zip	Gender	
0	500	Ava	Kerr	ava.kerr@google.com	35A Cherry Orchard	Denver	CO	80201	F	12
1	501	Victoria	Bailey	victoria.bailey@google.com	5 The Springs	Denver	CO	80201	F	25
2	502	Hannah	Mitchell	hannah.mitchell@google.com	79 Rydal Ave	Denver	CO	80201	F	85
3	503	Trevor	Lee	trevor.lee@google.com	101 Winchester Rd	Denver	CO	80201	M	95
4	504	Phil	Mackay	phil.mackay@google.com	12 Hylton Pl	Denver	CO	80201	M	95

In [23]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11 entries, 0 to 10
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype 
---  ------              --------------  ----- 
 0   Customer ID         11 non-null     int64 
 1   First Name          11 non-null     object
 2   Last Name           11 non-null     object
 3   Email Address       11 non-null     object
 4   Street Address      11 non-null     object
 5   City                11 non-null     object
 6   State               11 non-null     object
 7   Zip                 11 non-null     int64 
 8   Gender              11 non-null     object
 9   Credit Card Number  11 non-null     int64 
dtypes: int64(3), object(7)
memory usage: 1012.0+ bytes
```

In [24]:
```
df
```

Out[24]:

	Customer ID	First Name	Last Name	Email Address	Street Address	City	State	Zip
0	500	Ava	Kerr	ava.kerr@google.com	35A Cherry Orchard	Denver	CO	80201
1	501	Victoria	Bailey	victoria.bailey@google.com	5 The Springs	Denver	CO	80201
2	502	Hannah	Mitchell	hannah.mitchell@google.com	79 Rydal Ave	Denver	CO	80201
3	503	Trevor	Lee	trevor.lee@google.com	101 Winchester Rd	Denver	CO	80201
4	504	Phil	Mackay	phil.mackay@google.com	12 Hylton Pl	Denver	CO	80201
5	505	Abigail	Russell	abigail.russell@google.com	3 Ash Cl	Denver	CO	80201
6	506	Keith	Hill	keith.hill@google.com	28 Trewarton Rd	Denver	CO	80201
7	507	Neil	Underwood	neil.underwood@google.com	111 Clarence Rd	Denver	CO	80201
8	508	Sonia	Wright	sonia.wright@google.com	43 Emerson Ave	Denver	CO	80201
9	509	Carl	Jones	carl.jones@google.com	10 Brynawelon Dr	Denver	CO	80201
10	510	Bernadette	Chapman	bernadette.chapman@google.com	1 Central Circle	Denver	CO	80201

PART II PRACTICE QUESTIONS

1. Bring in the following csv files as tables in the feline database:

- Items.csv
- Orders.csv
- OrderLineItem.csv

2. View each table as a pandas dataframe to verify you successfully imported the data
3. Write the SQL statements necessary to answer the following questions, return the output in a pandas dataframe
   A. What are the e-mail addresses of the male customers?
   B. How many monthly items cost less than $20?
   C. List the customer first and last name alongside each order number.

4. Write code to create two new tables in the database one containing the order line items in orderLineItem.csv that are for monthly items, and another for those that are non-monthly items. Use a loop with f strings, then verify the success of the code by reading the tables back in from the database file/connection.

In [25]:
```
# Bring in the following csv files as tables in the feline database: Items.csv, Orders.csv, and (
'''YOUR CODE HERE'''
tableNames = ['Items','Orders','OrderLineItem']
for name in tableNames:
    df = pd.read_csv(name + '.csv')
    df.to_sql(name,connection,if_exists='replace',index=False)
```

In [26]:
```
# View each table as a pandas dataframe to verify you successfully imported the data
'''YOUR CODE HERE'''
# Make a query getting everything
for name in tableNames:
    print(20*'%',name,50*'%')
    getAllQuery = 'SELECT * from ' + name + ';'
    print(pd.read_sql_query(getAllQuery,connection))
```

%%%%%%%%%%%%%%%%%%%% Items %%%%%%%%%%%%%%%%%%%%

```
14       481      908         1
15       482      905         2
16       482      908         1
17       482      909         1
18       483      906         2
19       483      908         1
20       483      909         1
21       484      906         1
22       484      907         1
23       484      908         1
24       484      909         1
25       485      907         2
26       485      908         1
27       486      908         1
28       486      909         1
29       487      905         2
30       487      908         1
31       487      909         1
32       488      905         1
33       488      907         3
34       488      908         1
35       488      909         1
36       489      906         1
37       489      907         1
38       489      908         1
39       489      909         1
```

In [27]:
```
# A. What are the e-mail addresses of the male customers?
'''YOUR CODE HERE'''
sql1 = """
SELECT [Email Address]
FROM Customers
WHERE gender = 'M';
"""
pd.read_sql_query(sql1,connection)
```

Out[27]:

	Email Address
0	trevor.lee@google.com
1	phil.mackay@google.com
2	keith.hill@google.com
3	neil.underwood@google.com
4	carl.jones@google.com

In [28]:
```
# B. How many monthly items cost less than $20?
'''YOUR CODE HERE'''
sql2 = """
SELECT Count([Monthly?]) AS [Under $20 Monthly Items]
FROM Items
WHERE Price < 20 and [Monthly?]=1
;
"""
df = pd.read_sql_query(sql2,connection)
df
```

Out[28]:

	Under $20 Monthly Items
0	3

In [29]:
```
# C. List the customer first and last name alongside each order number.
'''YOUR CODE HERE'''
sql3 = """
SELECT [First Name],[Last Name],[Order ID]
FROM Customers
INNER JOIN Orders
ON Orders.[Customer ID] = Customers.[Customer ID]
;
"""
df = pd.read_sql_query(sql3,connection)
df
```

Out[29]:

	First Name	Last Name	Order ID
0	Victoria	Bailey	470
1	Victoria	Bailey	480
2	Hannah	Mitchell	471
3	Hannah	Mitchell	481
4	Trevor	Lee	472
5	Trevor	Lee	482
6	Phil	Mackay	473
7	Phil	Mackay	483
8	Abigail	Russell	474
9	Abigail	Russell	484
10	Keith	Hill	475
11	Keith	Hill	485
12	Neil	Underwood	476
13	Neil	Underwood	486
14	Sonia	Wright	477
15	Sonia	Wright	487
16	Carl	Jones	478
17	Carl	Jones	488
18	Bernadette	Chapman	489

In [30]:
```
""" 4. Write code to create two new tables in the database one containing the order line item rov
and another for those that are non-monthly items. Use a loop with f strings, then verify the succ
tables back in from the database file/connection."""

# Start by reading the orders and items tables into a data frame again
```

```
orderlineitems_df = pd.read_sql_query('SELECT * from OrderLineItem',connection)
items_df = pd.read_sql_query('SELECT * from Items',connection)
print(orderlineitems_df)
print(items_df)
```

```
    Order ID  Item ID  Quantity
0        470      901         1
1        471      902         2
2        472      901         1
3        473      902         3
4        474      903         2
5        475      904         3
6        476      901         1
7        477      902         4
8        478      903         2
9        480      905         2
10       480      908         1
11       480      909         1
12       481      906         2
13       481      907         2
14       481      908         1
15       482      905         2
16       482      908         1
17       482      909         1
18       483      906         2
19       483      908         1
20       483      909         1
21       484      906         1
22       484      907         1
23       484      908         1
24       484      909         1
25       485      907         2
26       485      908         1
27       486      908         1
28       486      909         1
29       487      905         2
30       487      908         1
31       487      909         1
32       488      905         1
33       488      907         3
34       488      908         1
35       488      909         1
36       489      906         1
37       489      907         1
38       489      908         1
39       489      909         1
   Item ID              Item Name  \
0      901     Single Cat Monthly
1      902  Multiple Cats Monthly
2      903       Toy of the month
3      904     Treat of the month
4      905         Organic litter
5      906              Cat Condo
6      907              Cat bowls
7      908              Cat brush
8      909              Flea comb

                                        Description   Price  Monthly?
0                 Monthly box of cat toys and treats   19.99         1
1  Monthly box of cat toys and treats for multipl...   35.99         1
2                            Monthly unique cat toy    9.99         1
3                  Monthly mouth-watering cat treats    4.59         1
4          Totally organic and fragrance free litter   25.00         0
5                 6 foot tall condo with 3 platforms  219.99         0
```

```
6                Multi-colored bowls with cat faces   18.99         0
7                             Two sided cat brush   12.50         0
8                Fine-toothed comb for flea control    5.99         0
```

In [31]:
```
# Next set up queries to get the monthly items only or non-monthly items only
# And write to a table name Monthly or Non_Monthly accordingly
for monthly in [1,0]:
    sql_command = f"""SELECT [Order ID],Items.[Item ID], [Quantity],  [Item Name], [Description],
                    FROM OrderLineItem
                    JOIN Items
                    on OrderLineItem.[Item ID] = Items.[Item ID]
                    WHERE Items.[Monthly?] = {monthly}; """
    df = pd.read_sql_query(sql_command, connection)
    table_name = 'Non_' * (1-monthly) + 'Monthly'
    df.to_sql(table_name, connection, if_exists='replace',index=False)
    print(df.head())
```

```
   Order ID  Item ID  Quantity              Item Name  \
0       470      901         1     Single Cat Monthly
1       472      901         1     Single Cat Monthly
2       476      901         1     Single Cat Monthly
3       471      902         2  Multiple Cats Monthly
4       473      902         3  Multiple Cats Monthly

                                      Description  Price
0                Monthly box of cat toys and treats  19.99
1                Monthly box of cat toys and treats  19.99
2                Monthly box of cat toys and treats  19.99
3  Monthly box of cat toys and treats for multipl...  35.99
4  Monthly box of cat toys and treats for multipl...  35.99
   Order ID  Item ID  Quantity       Item Name  \
0       480      905         2  Organic litter
1       482      905         2  Organic litter
2       487      905         2  Organic litter
3       488      905         1  Organic litter
4       481      906         2       Cat Condo

                                Description   Price
0  Totally organic and fragrance free litter   25.00
1  Totally organic and fragrance free litter   25.00
2  Totally organic and fragrance free litter   25.00
3  Totally organic and fragrance free litter   25.00
4         6 foot tall condo with 3 platforms  219.99
```

In [32]:
```
# Run this command to display names of all the tables in your database
cursor.execute("""SELECT name
                from sqlite_master
                where type = 'table';""")
print(cursor.fetchall())
```

```
[('Customers',), ('Items',), ('Orders',), ('OrderLineItem',), ('Monthly',), ('Non_Monthly',)]
```

Now check to see if the data was written to the tables by reading it back in from the tables. Loop through the monthly or not possibilities, creating sql queries via f strings to select all records from each table and display the results for each. Each record should appear only once in each table (or dataframe).

In [33]:
```
for monthly in [0,1]:
    table_name = 'Non_' * (1-monthly) + 'Monthly'
```

```
    sql_command = f"""SELECT *
                FROM {table_name}
                ;"""
    df = pd.read_sql_query(sql_command, connection)
    print(table_name)
    print(df)
```

Non_Monthly

	Order ID	Item ID	Quantity	Item Name	Description	Price
0	480	905	2	Organic litter	Totally organic and fragrance free litter	25.00
1	482	905	2	Organic litter	Totally organic and fragrance free litter	25.00
2	487	905	2	Organic litter	Totally organic and fragrance free litter	25.00
3	488	905	1	Organic litter	Totally organic and fragrance free litter	25.00
4	481	906	2	Cat Condo	6 foot tall condo with 3 platforms	219.99
5	483	906	2	Cat Condo	6 foot tall condo with 3 platforms	219.99
6	484	906	1	Cat Condo	6 foot tall condo with 3 platforms	219.99
7	489	906	1	Cat Condo	6 foot tall condo with 3 platforms	219.99
8	481	907	2	Cat bowls	Multi-colored bowls with cat faces	18.99
9	484	907	1	Cat bowls	Multi-colored bowls with cat faces	18.99
10	485	907	2	Cat bowls	Multi-colored bowls with cat faces	18.99
11	488	907	3	Cat bowls	Multi-colored bowls with cat faces	18.99
12	489	907	1	Cat bowls	Multi-colored bowls with cat faces	18.99
13	480	908	1	Cat brush	Two sided cat brush	12.50
14	481	908	1	Cat brush	Two sided cat brush	12.50
15	482	908	1	Cat brush	Two sided cat brush	12.50
16	483	908	1	Cat brush	Two sided cat brush	12.50
17	484	908	1	Cat brush	Two sided cat brush	12.50
18	485	908	1	Cat brush	Two sided cat brush	12.50
19	486	908	1	Cat brush	Two sided cat brush	12.50
20	487	908	1	Cat brush	Two sided cat brush	12.50
21	488	908	1	Cat brush	Two sided cat brush	12.50
22	489	908	1	Cat brush	Two sided cat brush	12.50
23	480	909	1	Flea comb	Fine-toothed comb for flea control	5.99
24	482	909	1	Flea comb	Fine-toothed comb for flea control	5.99
25	483	909	1	Flea comb	Fine-toothed comb for flea control	5.99
26	484	909	1	Flea comb	Fine-toothed comb for flea control	5.99
27	486	909	1	Flea comb	Fine-toothed comb for flea control	5.99
28	487	909	1	Flea comb	Fine-toothed comb for flea control	5.99
29	488	909	1	Flea comb	Fine-toothed comb for flea control	5.99
30	489	909	1	Flea comb	Fine-toothed comb for flea control	5.99

```
24         Fine-toothed comb for flea control    5.99
25         Fine-toothed comb for flea control    5.99
26         Fine-toothed comb for flea control    5.99
27         Fine-toothed comb for flea control    5.99
28         Fine-toothed comb for flea control    5.99
29         Fine-toothed comb for flea control    5.99
30         Fine-toothed comb for flea control    5.99
Monthly
   Order ID  Item ID  Quantity              Item Name  \
0       470      901         1     Single Cat Monthly
1       472      901         1     Single Cat Monthly
2       476      901         1     Single Cat Monthly
3       471      902         2  Multiple Cats Monthly
4       473      902         3  Multiple Cats Monthly
5       477      902         4  Multiple Cats Monthly
6       474      903         2       Toy of the month
7       478      903         2       Toy of the month
8       475      904         3     Treat of the month

                                        Description  Price
0                 Monthly box of cat toys and treats  19.99
1                 Monthly box of cat toys and treats  19.99
2                 Monthly box of cat toys and treats  19.99
3  Monthly box of cat toys and treats for multipl...  35.99
4  Monthly box of cat toys and treats for multipl...  35.99
5  Monthly box of cat toys and treats for multipl...  35.99
6                            Monthly unique cat toy   9.99
7                            Monthly unique cat toy   9.99
8                 Monthly mouth-watering cat treats   4.59
```

In [34]:
```
# Close everything
connection.close()
```

In [ ]:
```

```

In [ ]:
```

```